

Network Traffic Analysis Report

Home Lab — Wireshark Packet Capture & Analysis

Project 4 | Analyst: Chaithanya Thota | Date: June 11, 2026

1. Background and Purpose

This report documents a hands-on network traffic analysis exercise conducted in a home cybersecurity lab. The goal of this project was to set up a two-machine virtual lab environment, generate real network traffic between the machines, and use Wireshark to capture, filter, and analyze that traffic — replicating the workflow used by Security Operations Centre (SOC) analysts and network security professionals every day.

The lab was built entirely using free, open-source tools running inside VirtualBox virtual machines. No real network infrastructure was needed, and all traffic was contained within the virtual environment — making this a safe and legal exercise.

This type of hands-on lab work is considered essential preparation for roles such as SOC Analyst, Network Security Engineer, and Incident Responder, where the ability to read and interpret packet captures is a core day-one skill.

2. Lab Environment Setup

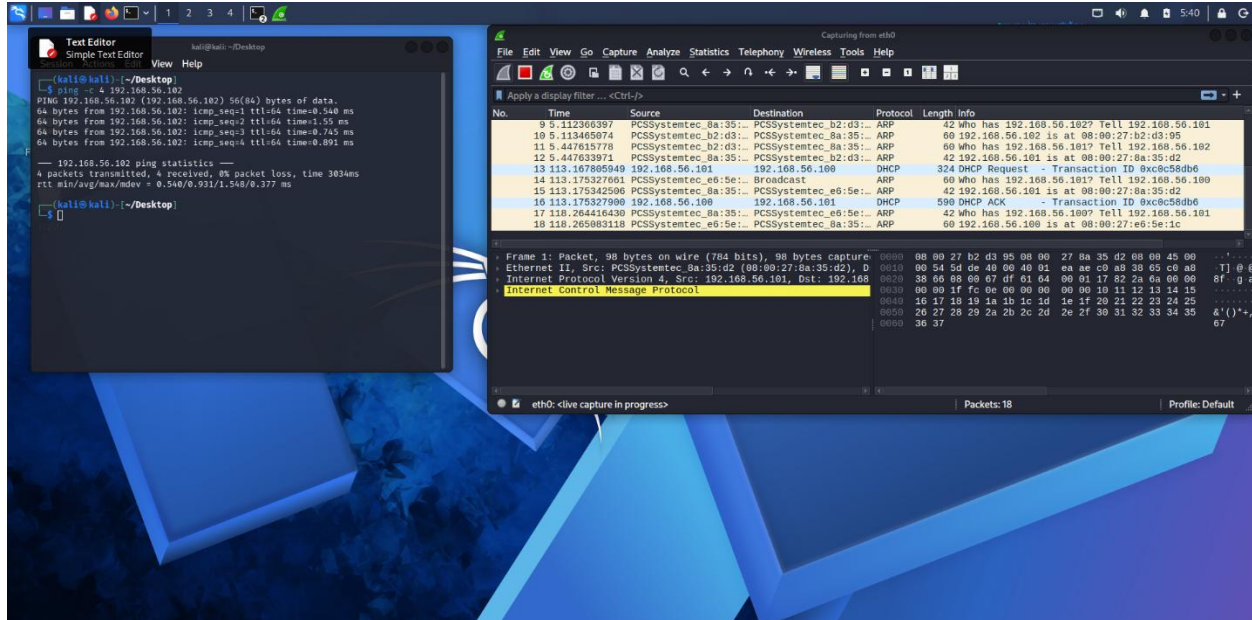
The lab consisted of two virtual machines running simultaneously on the same physical host, connected through a VirtualBox Host-Only network adapter. This configuration creates an isolated virtual network that allows the two machines to communicate with each other without any traffic reaching the real internet.

Lab Configuration	
Machine 1 (Attacker)	Kali Linux 2026.1 — IP: 192.168.56.101
Machine 2 (Target)	Ubuntu Server 24.04 LTS — IP: 192.168.56.102
Network Type	VirtualBox Host-Only Adapter — 192.168.56.0/24
Capture Tool	Wireshark 4.6.6 running on Kali Linux
Capture Interface	eth0 — the Host-Only network adapter
Traffic Generated	ICMP (ping), ARP, DHCP, TCP (Nmap port scan)
Total Packets	2,035+ packets captured during the session

Capture File

wireshark_eth05VTRQ3.pcapng

The network diagram is straightforward: Kali Linux at `192.168.56.101` acts as the analyst machine where Wireshark runs and network scans originate. Ubuntu Server at `192.168.56.102` acts as the target machine. All traffic between the two machines passes through the virtual network adapter and is visible to Wireshark.



3. Traffic Types Captured and Explained

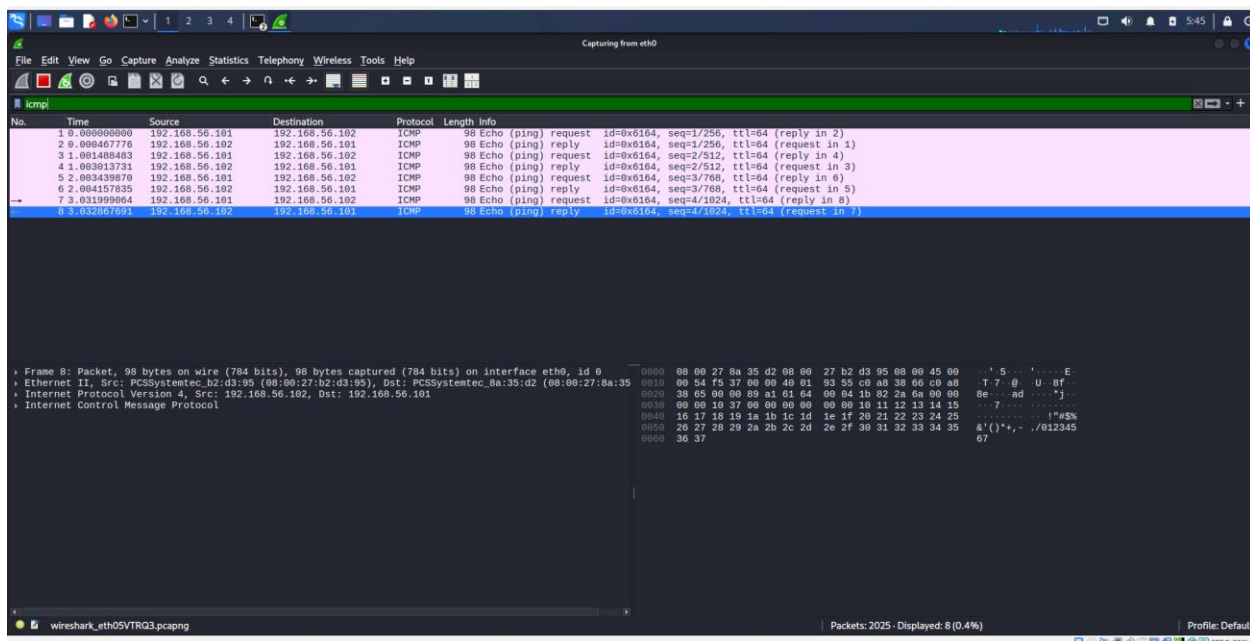
During the capture session, four distinct types of network traffic were observed. Each one tells a different story about what was happening on the network at that moment.

3.1 ARP Traffic — Finding Each Other

ARP stands for Address Resolution Protocol. Before any two devices can communicate on a network, they need to know each other's MAC address — the physical hardware address of the network card. ARP is the mechanism they use to find out.

In the capture, the very first thing that appeared when the ping was run was an ARP request from Kali asking "Who has 192.168.56.102? Tell 192.168.56.101." Ubuntu responded with its MAC address, and then the actual ping could begin. This is like knocking on a door and asking "is this the right house?" before sending a letter.

ARP traffic is important from a security perspective because it can be abused in an attack called ARP Poisoning or ARP Spoofing — where an attacker sends fake ARP replies to redirect traffic through their machine. Seeing unexpected ARP replies in a capture is a classic indicator of this type of attack.

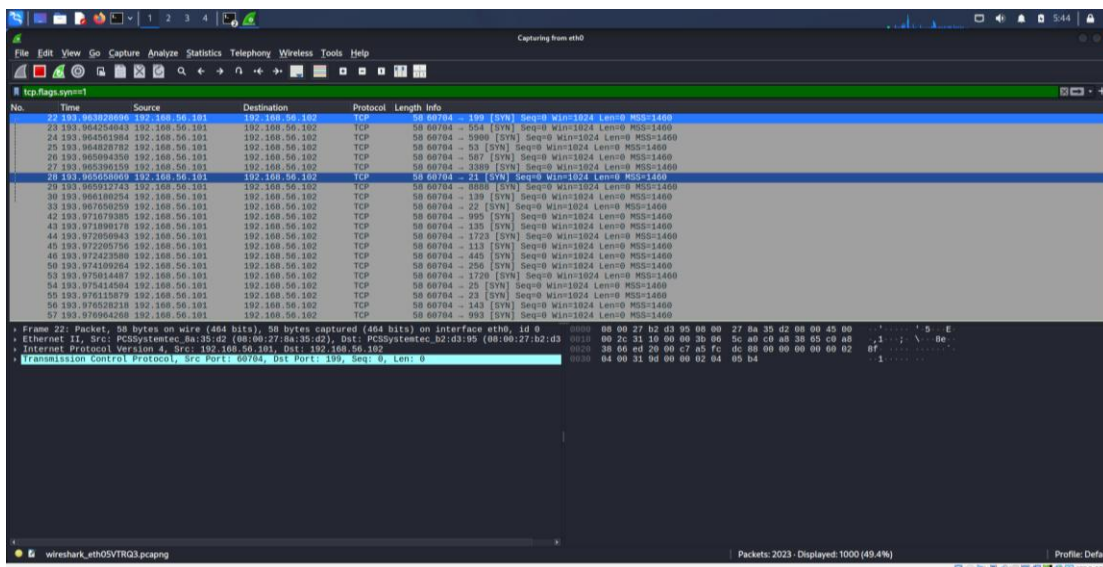


3.2 ICMP Traffic — The Ping Test

ICMP stands for Internet Control Message Protocol. The ping command uses ICMP to test connectivity between two machines. Each ping sends an ICMP Echo Request packet, and if the target is reachable it responds with an ICMP Echo Reply.

In the capture, running `ping -c 4 192.168.56.102` produced exactly 8 ICMP packets — 4 requests from Kali and 4 replies from Ubuntu. Each packet was 98 bytes in size. The round-trip time ranged from 0.54ms to 1.55ms, which is extremely fast because both machines are on the same physical host.

From a security analyst perspective, a sudden flood of ICMP traffic in a real network could indicate a ping sweep — where an attacker is trying to discover which machines are alive on a network. This is often the first phase of a reconnaissance attack.



3.3 DHCP Traffic — Getting an IP Address

DHCP stands for Dynamic Host Configuration Protocol. When Ubuntu Server started up and connected to the Host-Only network, it automatically requested an IP address from the DHCP server. This process was captured in Wireshark and produced two clearly visible packets.

The first packet was a DHCP Request from Ubuntu (showing as 192.168.56.100 broadcast) asking for an IP address. The second was a DHCP ACK (acknowledgement) confirming the assignment of 192.168.56.102. This is the same process that happens every time you connect your phone to a WiFi network — the phone asks the router for an IP address and the router assigns one.

Analysing DHCP traffic is useful for detecting rogue DHCP servers on a network — a common attack where a malicious device starts handing out fake IP addresses to redirect traffic.

3.4 TCP Traffic — The Nmap Port Scan

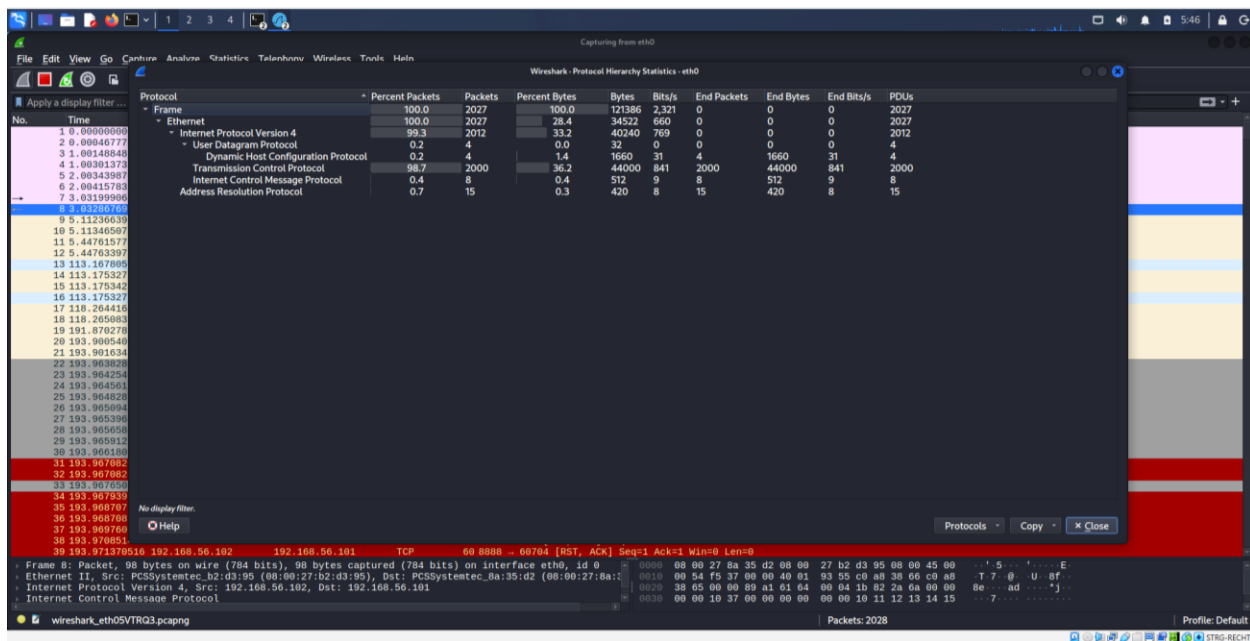
The most dramatic traffic in the capture came from running `nmap 192.168.56.102` from Kali. Nmap is a port scanner — it sends a TCP connection attempt to every single port on the target machine to find out which ones are open and which services are running.

This generated over 2,000 packets in Wireshark, all following the same pattern. Kali sent TCP SYN packets (the first step of a connection — like knocking on a door) to each of the 1,000 ports it tested. Ubuntu responded to every single one with a TCP RST ACK packet (the equivalent of slamming the door shut and saying "nobody home"). This is because Ubuntu Server had no services running and all ports were closed.

The Nmap results confirmed this finding:

- 1,000 ports scanned in 0.38 seconds
- All 1,000 ports returned as closed (reset)
- No open services detected
- Ubuntu's MAC address identified as an Oracle VirtualBox virtual NIC

From a detection perspective, this traffic pattern — thousands of TCP SYN packets from one source to many different ports in rapid succession — is one of the most recognisable signatures of a port scan. Any modern Intrusion Detection System (IDS) would immediately flag this as suspicious activity.



```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ ping -c 4 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data:
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.540 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=1.55 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=0.745 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=0.891 ms

— 192.168.56.102 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3034ms
rtt min/avg/max/mdev = 0.540/0.931/1.548/0.377 ms

(kali@kali)-[~/Desktop]
$ nmap 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-11 05:41 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.102
Host is up (0.0035s latency).
All 1000 scanned ports on 192.168.56.102 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 08:00:27:B2:D3:95 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.38 seconds

(kali@kali)-[~/Desktop]
$
```


4. Key Findings and Analysis

Looking at the full capture session as a whole, several important observations stand out from an analyst perspective.

- **Finding 1:** Ubuntu Server was well secured at time of scan — all 1,000 ports were closed with no services exposed. In a real environment, an attacker running this scan would find nothing to exploit.
- **Finding 2:** The port scan was easily detectable — the pattern of 2,000+ SYN packets in under half a second is an unmistakable signature. Any organisation running an IDS or monitoring Wireshark captures would immediately see this.
- **Finding 3:** ARP traffic reveals the network topology — by analysing ARP packets alone, an analyst can map out which devices are present on a network and what their MAC addresses are, even without seeing any application-layer traffic.
- **Finding 4:** DHCP traffic reveals when devices join the network — the timestamp of DHCP requests tells an analyst exactly when Ubuntu Server was started and connected. This is useful for building a timeline during an incident investigation.
- **Finding 5:** The speed of modern tools is striking — Nmap scanned 1,000 ports in 0.38 seconds. On a real network, an attacker can scan an entire organisation's IP range in minutes. This highlights why real-time monitoring is essential.

5. What This Looks Like in the Real World

The traffic patterns captured in this lab exercise are identical to what SOC analysts see every day in real corporate environments. The key difference is scale — a real network might have thousands of machines generating millions of packets per hour rather than two machines generating a few thousand.

A real SOC analyst using Wireshark or a similar tool (such as Zeek or Suricata) would apply the same filters and techniques used in this exercise to isolate suspicious traffic from the background noise. The ability to spot a port scan, identify an ARP spoofing attack, or trace a specific connection through a packet capture is a fundamental skill that this lab directly develops.

The .pcap file produced in this exercise is also a standard artefact in incident response. When a security incident occurs, analysts often capture network traffic for forensic analysis — examining exactly what data was sent, to where, and when. The file format used here (pcapng) is the industry standard and can be opened by any network analysis tool.

6. Tools and Commands Used

Tool / Command	Purpose	What It Showed
Wireshark 4.6.6	Packet capture & analysis	Live view of all network traffic between both VMs
<code>ping -c 4 [IP]</code>	Connectivity test	Generated ICMP traffic — confirmed both machines can communicate
<code>nmap [IP]</code>	Port scanner	Generated TCP SYN traffic — revealed all ports closed on Ubuntu
<code>ip a</code>	IP address check	Confirmed IP assignments on both machines
<code>tcp.flags.syn==1</code>	Wireshark filter	Isolated all SYN packets to show Nmap scan pattern clearly
<code>icmp</code>	Wireshark filter	Isolated ping packets from all other traffic
<code>arp</code>	Wireshark filter	Showed address resolution between the two machines

7. Conclusion

This project successfully demonstrated the full workflow of network traffic capture and analysis using a realistic two-machine lab setup. Starting from scratch — setting up two virtual machines, configuring them to communicate, and capturing live traffic — mirrors the environment a SOC analyst works in every day.

The most important lesson from this exercise is that network traffic tells a story. ARP packets reveal who is on the network and when they arrived. ICMP packets reveal connectivity tests and potential reconnaissance. TCP SYN floods reveal port scanning activity. Each protocol has a signature, and learning to recognise those signatures is the foundation of network-based threat detection.

The .pcap file produced in this exercise is a permanent record of everything that happened on the virtual network during the capture session. In a real incident, this type of file is used as forensic evidence — analysed to reconstruct exactly what an attacker did, when they did it, and what they accessed. Knowing how to capture and read these files is a skill that directly applies on day one of a SOC analyst role.